

SBA301 — SLOT 18

BÀI TẬP / DỰ ÁN TỔNG HỢP

LAB 04 — ORCHID REST API & JPA INTEGRATION KIT

Chapter 13 Part A + Part B • Entity • Relationship • Repository • Service • REST Controller • SQL Server • Postman

Mental model: HTTP Request → REST Controller → Service / Transaction → JpaRepository → Hibernate / JPA → SQL Server → Entity/Relationship → HTTP Response → Postman Verification.

Thông tin	Giá trị
Course	SBA301 – Integrate Single Page Application with Spring Boot
Slot	18 – Chapter 13 Part B + Lab 04; củng cố Part A của Slot 17
Bài tập	Orchid REST API & JPA Integration Kit
Độ khó	Trung bình → Khá
Thời lượng	3–5 giờ thực hành + hoàn thiện evidence ngoài lớp
Hình thức	Cá nhân hoặc nhóm nhỏ; mỗi sinh viên phải giải thích được flow
Công cụ chính	JDK 21, IntelliJ IDEA, Spring Boot 3.x, Maven, Spring Data JPA, SQL Server, Postman
Trọng tâm	JPA mapping, relationship, repository, service/transaction, REST controller, CRUD, status code, verification
Không thuộc core	React frontend, authentication/authorization, MongoDB, microservices, deployment production

Chu trình thực hành: BASELINE → CONFIGURE DB → MAP ENTITY/RELATIONSHIP → REPOSITORY → SERVICE → CONTROLLER → RUN → CRUD TEST → VERIFY DB/SQL → BREAK/FIX → DOCUMENT → EXPLAIN.

0. THÔNG TIN DỰ ÁN

Hạng mục	Nội dung
Tên dự án	Orchid REST API & JPA Integration Kit
Mục đích	Củng cố Chapter 13 Part A (JPA/Entity/Repository/Service/Relationships) và Part B (REST Controller + REST/JPA integration).
Resource chính	Orchid
Relationship bắt buộc	Orchid N–1 OrchidCategory; OrchidCategory 1–N Orchid
Database	Microsoft SQL Server
API client	Postman
Thời lượng	3–5 giờ
Sản phẩm nộp	Spring Boot project, SQL evidence, Postman collection/evidence, test-matrix.md, README.md, concept-trace.md

Ranh giới kiến thức: Bài này tập trung backend Spring Boot + JPA + REST. Không dành thời gian dựng React UI. Frontend sẽ chỉ được xem là client tương lai sử dụng API contract đã kiểm chứng.

1. TÓM TẮT DỰ ÁN VÀ MỤC TIÊU

Sinh viên xây dựng RESTful API cho hệ thống Orchid Management. Dữ liệu Orchid được lưu bền vững trong SQL Server bằng Spring Data JPA. Mỗi Orchid thuộc một OrchidCategory. Sinh viên phải triển khai đủ Entity/Relationship → Repository → Service → REST Controller, chạy ứng dụng, kiểm thử CRUD bằng Postman, đọc SQL/DDDL được Hibernate sinh ra và xác minh trạng thái dữ liệu sau mỗi thao tác.

1.1 Mục tiêu kiến thức

- Phân biệt JPA, Hibernate và Spring Data JPA trong một request thực tế.
- Giải thích vai trò của @Entity, @Id, @GeneratedValue, @Column, @ManyToOne, @OneToMany, mappedBy và @JoinColumn.
- Xác định owning side của quan hệ Orchid N–1 OrchidCategory và liên hệ với foreign key category_id.
- Giải thích vì sao JpaRepository không cần tự viết implementation CRUD cơ bản.
- Giải thích Service là nơi đặt business rule và transaction boundary thay vì đưa toàn bộ logic vào Controller.
- Mapping CRUD với GET/POST/PUT/DELETE và status code phù hợp.
- Trace được HTTP request từ Controller đến Service, Repository, Hibernate, SQL Server và quay lại response JSON.
- Phân biệt lỗi route, missing resource, missing relationship và lỗi kết nối database.

1.2 Mục tiêu kỹ năng

- Tạo/cấu hình Spring Boot 3.x project với Spring Web, Spring Data JPA, SQL Server Driver và Test.
- Kết nối SQL Server bằng application.properties mà không hard-code credential vào tài liệu nộp công khai.
- Thiết kế hai entity có quan hệ 1–N/N–1 và kiểm chứng schema/foreign key.
- Tạo repository, derived query đơn giản và service CRUD có transaction phù hợp.
- Tạo REST Controller với @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PathVariable và @RequestParam.
- Tạo Postman collection, chạy positive/negative tests và kiểm tra hậu điều kiện.
- Thu thập evidence: endpoint, status, response, SQL log và dữ liệu trong SQL Server.
- Viết README, test matrix và concept trace để chứng minh hiểu kiến trúc chứ không chỉ có code chạy.

Kết quả đầu ra: Bài hoàn thành khi API chạy với SQL Server thật, CRUD được kiểm thử, relationship được kiểm chứng ở cả Java object graph và foreign key, evidence đầy đủ, và sinh viên có thể giải thích một request end-to-end mà không đọc code từng dòng.

2. CHỨC NĂNG CỦA BÀI TẬP

#	Chức năng	Kết quả chính	Kiến thức củng cố
1	Project baseline	Spring Boot project build được	Build/tooling
2	SQL Server connection	Datasource hoạt động	JDBC/DataSource
3	Category Entity	Bảng orchid_categories	@Entity, PK
4	Orchid Entity	Bảng orchids	Entity mapping
5	N-1 / 1-N mapping	category_id FK	@ManyToOne/@OneToMany
6	Repository	JpaRepository interfaces	Repository abstraction
7	Derived query	Search by orchidName	Query method
8	Service	CRUD + relationship validation	Business logic
9	Transaction boundary	Write operations atomic	@Transactional
10	REST Controller	/api/orchids	REST mapping
11	GET list/detail	200 / 404	GET, @PathVariable
12	GET search	?name=...	@RequestParam
13	POST create	201 + FK category	Request body
14	PUT update	200 + full state update	PUT semantics
15	DELETE	204 + verify missing	DELETE + postcondition
16	SQL/DB inspection	Rows + FK + generated SQL	JPA→SQL verification
17	Break-It Lab	Có lỗi → chẩn đoán → sửa	Debugging
18	Evidence package	Test matrix + screenshots	Verification
19	Concept trace	Request-to-database explanation	Architecture understanding
20	AI verification	Critique + human verification	AI-assisted learning

3. CÔNG NGHỆ VÀ CÔNG CỤ SỬ DỤNG

Công nghệ / công cụ	Vai trò	Dùng trong bài
JDK 21	Java runtime/toolchain	Compile/run Spring Boot
Spring Boot 3.x	Application framework	Auto-configuration, embedded server
Spring Web / MVC	REST layer	Controller + HTTP mapping
Spring Data JPA	Persistence abstraction	JpaRepository + JPA integration
Hibernate	JPA provider	ORM, persistence context, SQL generation
Microsoft SQL Server	Relational database	Persistent tables, PK/FK
Maven	Build/dependency management	pom.xml, test, package
IntelliJ IDEA	IDE	Code, terminal, run/debug
Postman	API client	CRUD + negative tests + evidence

Công nghệ / công cụ	Vai trò	Dùng trong bài
Git	Checkpoint/reproducibility	Commit/tag milestone
Markdown	Documentation	README, test matrix, concept trace

Phiên bản: dùng JDK 21 và Spring Boot 3.x theo course track. Không pin version riêng cho starter do Spring Boot dependency management quản lý. Với SQL Server Driver, ưu tiên dependency do Spring Initializr tạo.

4. CẤU TRÚC THƯ MỤC DỰ ÁN

Cấu trúc đề xuất

```

slot18-orchid-jpa-rest-lab/
├── docs/
│   ├── api-contract.md
│   ├── concept-trace.md
│   └── test-matrix.md
├── evidence/
│   ├── postman/
│   ├── sqlserver/
│   └── console/
├── src/
│   ├── main/
│   │   ├── java/com/example/orchid/
│   │   │   ├── OrchidApplication.java
│   │   │   ├── controllers/OrchidController.java
│   │   │   ├── pojos/Orchid.java
│   │   │   ├── pojos/OrchidCategory.java
│   │   │   ├── repositories/IOrchidRepository.java
│   │   │   ├── repositories/IOrchidCategoryRepository.java
│   │   │   ├── services/IOrchidService.java
│   │   │   └── services/OrchidService.java
│   │   └── resources/application.properties
│   └── test/
├── pom.xml
└── README.md

```

Thành phần	Mục đích
pojos/	JPA entities và relationship mapping.
repositories/	Data access boundary bằng Spring Data JPA.
services/	Business logic, relationship validation, transaction boundary.
controllers/	HTTP/REST boundary, status code, path/query/body mapping.
docs/api-contract.md	Method, endpoint, request, response, status code.
docs/test-matrix.md	Positive/negative test và kết quả thực tế.
docs/concept-trace.md	Trace request → controller → service → repository → SQL → response.
evidence/	Ảnh Postman, SQL Server, console/SQL log theo checkpoint.
README.md	Cài đặt, cấu hình, chạy, test, known issues và phạm vi lab.

5. YÊU CẦU CÀI ĐẶT VÀ CHUẨN BỊ MÔI TRƯỜNG

5.1 Pre-flight checklist

- JDK 21 và Maven hoạt động: `java -version`, `mvn -version`.
- IntelliJ import được Maven project, không còn lỗi từ Lab 03.
- SQL Server service đang chạy và sinh viên có quyền tạo database hoặc được cấp database riêng.
- Postman sẵn sàng; có thể tạo collection variable baseUrl.

- Đã commit baseline trước khi chỉnh JPA/SQL Server: gợi ý tag s18-before-lab04.
- Không đưa password thật lên Git repository công khai.

Lệnh kiểm tra

```
java -version
mvn -version
git status
```

5.2 Dependencies bắt buộc

- Spring Web
- Spring Data JPA
- MS SQL Server Driver
- Spring Boot Starter Test

Có thể chọn dependency trực tiếp bằng Spring Initializr. Nếu chỉnh pom.xml thủ công, reload Maven và build lại trước khi viết Entity để tránh trộn lỗi dependency với lỗi mapping.

5.3 Database chuẩn bị

SQL Server – tạo database (ví dụ)

```
CREATE DATABASE OrchidDB;
GO
```

Credential: Trong file nộ, thay password bằng placeholder YOUR_PASSWORD hoặc mô tả cách cấu hình bằng biến môi trường. Không chụp màn hình để lộ password.

6. TÓM TẮT DANH SÁCH CÁC BƯỚC THỰC HIỆN

Bước	Nội dung	Milestone
1	Xác nhận baseline Lab 03 / project clean	V0
2	Tạo/import Spring Boot project cho Lab 04	V1
3	Kiểm tra pom.xml và Maven build	V1
4	Tạo database + application.properties	V2
5	Tạo package/project structure	V2
6	Tạo OrchidCategory Entity	V3
7	Tạo Orchid Entity + Many-to-One	V3
8	Hoàn thiện One-to-Many + kiểm tra owning side	V3
9	Tạo JPA repositories	V4
10	Thêm derived query search by name	V4
11	Tạo IOrchidService	V5
12	Implement OrchidService + transaction	V5
13	Implement OrchidController	V6
14	Chạy app + verify DDL/SQL/schema	V7
15	Tạo Postman collection + API contract	V7
16	GET list + search	V8

Bước	Nội dung	Milestone
17	GET detail + missing resource	V8
18	POST create + verify relationship	V8
19	PUT update + verify	V8
20	DELETE + verify postcondition	V8
21	Break-It JPA/REST lab	V9
22	Lập test matrix + evidence package	V9
23	Viết README + concept trace	V10
24	Final verification + Git checkpoint	V10

7. HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

Bước 1. Xác nhận baseline Lab 03 / project clean

Mục	Nội dung
Mục tiêu	Bắt đầu từ trạng thái Spring Boot REST project có thể build/run trước khi thêm persistence thật.
Thao tác	Checkout/open project baseline; chạy mvn test hoặc mvn clean test; kiểm tra git status.
Kết quả mong đợi	Build PASS; không có lỗi compilation; trạng thái Git được biết rõ.
Cách kiểm tra	Nếu baseline dùng in-memory repository, gọi một endpoint cũ để xác nhận REST layer hoạt động.
Lỗi thường gặp	Bắt đầu JPA trong khi project cũ đang lỗi; không commit baseline nên khó rollback.

Trạng thái project: V0 – Baseline ổn định.

Terminal

```
mvn clean test
git status
```

Mẹo: Nếu Lab 03 chưa ổn định, sửa build/test trước. Không thêm JPA để “hy vọng” lỗi cũ tự hết.

Bước 2. Tạo/import Spring Boot project cho Lab 04

Mục	Nội dung
Mục tiêu	Có project đúng course track JDK 21 + Spring Boot 3.x.
Thao tác	Dùng Spring Initializr trong IntelliJ hoặc start.spring.io. Chọn Maven/Java/JDK 21; đặt package gốc ví dụ com.example.orchid.
Kết quả mong đợi	Application class được tạo; project import Maven thành công.
Cách kiểm tra	Chạy class @SpringBootApplication một lần trước khi cấu hình database.
Lỗi thường gặp	Package root đặt sai làm component scan không thấy controller/service/repository.

Trạng thái project: V1 – Spring Boot skeleton chạy được.

Bước 3. Kiểm tra pom.xml và Maven build

Mục	Nội dung
Mục tiêu	Có đúng dependency cho REST + JPA + SQL Server + test.
Thao tác	Xác nhận pom.xml có spring-boot-starter-web, spring-boot-starter-data-jpa, mssql-jdbc và spring-boot-starter-test; Maven Reload.
Kết quả mong đợi	mvn clean test hoàn tất, dependency tree resolve được.
Cách kiểm tra	Mở Maven tool window hoặc chạy mvn dependency:tree khi cần.
Lỗi thường gặp	Tự thêm version cho starter không cần thiết; dùng nhầm javax.persistence thay vì jakarta.persistence.

Trạng thái project: V1 – Dependencies sẵn sàng.

Dependency tối thiểu

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>com.microsoft.sqlserver</groupId>
    <artifactId>mssql-jdbc</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Bước 4. Tạo database và cấu hình application.properties

Mục	Nội dung
Mục tiêu	Spring Boot kết nối được SQL Server và Hibernate có thể quản lý schema.
Thao tác	Tạo OrchidDB. Điền URL, username, password phù hợp môi trường cá nhân; bật show-sql để phục vụ verification.
Kết quả mong đợi	Application start không báo datasource/login error; Hibernate kết nối đúng OrchidDB.
Cách kiểm tra	Đọc startup log; nếu lỗi Login failed/connection refused thì xử lý connection trước.
Lỗi thường gặp	Sai port; SQL Server TCP/IP chưa bật; sai databaseName; commit password thật.

Trạng thái project: V2 – Datasource kết nối được.

application.properties – mẫu

```
spring.application.name=slot18-orchid-lab
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=OrchidDB;encrypt=true;trustServerCertificate=true
spring.datasource.username=sa
spring.datasource.password=YOUR_PASSWORD
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
server.port=8080
```

Mẹo: Nếu lớp dùng instance/port khác, chỉ thay cấu hình môi trường; không đổi code business vì lỗi kết nối.

Bước 5. Tạo package/project structure

Mục	Nội dung
Mục tiêu	Tách rõ Controller, Service, Repository và Entity để phản ánh 3-layer architecture.
Thao tác	Tạo packages pojos, repositories, services, controllers và docs/evidence như Phần 4.
Kết quả mong đợi	Tất cả package nằm dưới package gốc chứa OrchidApplication.
Cách kiểm tra	Dùng Project view xác nhận package path và package declaration.
Lỗi thường gặp	Đặt service/controller ra ngoài component-scan root; tạo package tên tùy ý nhưng import nhầm.

Trạng thái project: V2 – Cấu trúc project rõ ràng.

Bước 6. Tạo OrchidCategory Entity

Mục	Nội dung
Mục tiêu	Tạo entity phía “one” của quan hệ Category 1–N Orchid.
Thao tác	Tạo OrchidCategory với categoryId, categoryName. Dùng @Entity, @Table, @Id, @GeneratedValue, @Column.
Kết quả mong đợi	Project compile; Hibernate có metadata để tạo table orchid_categories.
Cách kiểm tra	IntelliJ Generate constructor không tham số + getters/setters.
Lỗi thường gặp	Thiếu @Id; dùng import javax.persistence.*; thiếu constructor không tham số.

Trạng thái project: V3 – Category entity hợp lệ.

OrchidCategory.java – phần cốt lõi

```
package com.example.orchid.pojos;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "orchid_categories")
public class OrchidCategory {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long categoryId;

    @Column(nullable = false, unique = true, length = 100)
    private String categoryName;

    @OneToMany(mappedBy = "orchidCategory")
    @JsonIgnore
    private List<Orchid> orchids = new ArrayList<>();

    public OrchidCategory() {}
    public OrchidCategory(String categoryName) { this.categoryName = categoryName; }

    // Generate getters/setters with IntelliJ: Code > Generate
}
```

Bước 7. Tạo Orchid Entity + Many-to-One

Mục	Nội dung
Mục tiêu	Map Orchid sang table và khai báo owning side chứa foreign key.

Mục	Nội dung
Thao tác	Tạo các field từ Lab 04 và thêm @ManyToOne + @JoinColumn(name="category_id").
Kết quả mong đợi	Hibernate có thể tạo bảng orchids với category_id.
Cách kiểm tra	Kiểm tra field name của mappedBy ở Category phải đúng chính xác "orchidCategory".
Lỗi thường gặp	Dùng @OneToMany ở phía Orchid; đặt JoinColumn ở cả hai phía; mappedBy sai tên field.

Trạng thái project: V3 – Orchid entity + owning side hoàn chỉnh.

Orchid.java – phần cốt lõi

```
package com.example.orchid.pojos;

import jakarta.persistence.*;

@Entity
@Table(name = "orchids")
public class Orchid {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long orchidID;

    @Column(nullable = false, length = 150)
    private String orchidName;

    private Boolean isNatural;

    @Column(length = 1000)
    private String orchidDescription;

    @ManyToOne(optional = false)
    @JoinColumn(name = "category_id", nullable = false)
    private OrchidCategory orchidCategory;

    private Boolean isAttractive;
    private String orchidURL;

    public Orchid() {}

    // Generate getters/setters with IntelliJ: Code > Generate
}
```

Bước 8. Hoàn thiện One-to-Many và kiểm tra owning side

Mục	Nội dung
Mục tiêu	Giải thích được relationship thay vì chỉ làm annotation compile.
Thao tác	Đối chiếu hai entity. Xác định Orchid.orchidCategory là owning side; OrchidCategory.orchids là inverse side mappedBy.
Kết quả mong đợi	Sinh viên vẽ được: orchid_categories(category_id PK) ← orchids(category_id FK).
Cách kiểm tra	Viết 3 dòng vào docs/concept-trace.md: owning side, foreign key, ý nghĩa mappedBy.
Lỗi thường gặp	Cho rằng mappedBy là tên column; dùng mappedBy="category_id"; thêm cascade ALL mà không hiểu hậu quả.

Trạng thái project: V3 – Relationship được giải thích và kiểm chứng bằng model.

Mẹo: mappedBy nhận TÊN FIELD Java ở owning side, không phải tên foreign-key column trong database.

Bước 9. Tạo JPA repositories

Mục	Nội dung
-----	----------

Mục	Nội dung
Mục tiêu	Có data access abstraction cho Orchid và OrchidCategory.
Thao tác	Tạo hai interface extends JpaRepository<Entity, Long>. Không tạo implementation CRUD thủ công.
Kết quả mong đợi	Application context có thể tạo repository beans.
Cách kiểm tra	Compile project; kiểm tra generic Entity/ID type khớp @Id.
Lỗi thường gặp	Dùng sai ID type; thêm @Repository là bắt buộc về mặt hiểu biết; viết OrchidRepositoryImpl không cần thiết.

Trạng thái project: V4 – Repository layer sẵn sàng.

Repositories

```
package com.example.orchid.repositories;

import com.example.orchid.pojos.Orchid;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface IOrchidRepository extends JpaRepository<Orchid, Long> {
    List<Orchid> findByOrchidNameContainingIgnoreCase(String name);
}
```

IOrchidCategoryRepository.java

```
package com.example.orchid.repositories;

import com.example.orchid.pojos.OrchidCategory;
import org.springframework.data.jpa.repository.JpaRepository;

public interface IOrchidCategoryRepository extends JpaRepository<OrchidCategory, Long> {}
```

Bước 10. Thêm derived query search by name

Mục	Nội dung
Mục tiêu	Củng cố semantics của Spring Data query method.
Thao tác	Giữ method findByOrchidNameContainingIgnoreCase(String name) trong IOrchidRepository.
Kết quả mong đợi	Spring Data tạo query dựa trên property orchidName.
Cách kiểm tra	Sau khi API hoàn thành, GET /api/orchids?name=cat phải lọc theo tên.
Lỗi thường gặp	Tên method không trùng property; viết SQL thủ công dù derived query đủ dùng.

Trạng thái project: V4 – Repository có CRUD + search.

Bước 11. Tạo IOrchidService

Mục	Nội dung
Mục tiêu	Xác định contract của business layer trước implementation.
Thao tác	Tạo interface với getAll, searchByName, getByName, create, update, delete.
Kết quả mong đợi	Controller sau này chỉ phụ thuộc service contract.
Cách kiểm tra	Không đưa ResponseEntity vào service interface.
Lỗi thường gặp	Service interface chứa HTTP concerns; controller gọi repository trực tiếp.

Trạng thái project: V5 – Service contract rõ ràng.

IOrchidService.java

```
package com.example.orchid.services;

import com.example.orchid.pojos.Orchid;
import java.util.List;
import java.util.Optional;

public interface IOrchidService {
    List<Orchid> getAll();
    List<Orchid> searchByName(String name);
    Optional<Orchid> getById(Long id);
    Orchid create(Orchid orchid);
    Optional<Orchid> update(Long id, Orchid orchid);
    boolean delete(Long id);
}
```

Bước 12. Implement OrchidService + transaction boundary

Mục	Nội dung
Mục tiêu	Đặt persistence orchestration và relationship validation vào Service.
Thao tác	Inject IOrchidRepository + IOrchidCategoryRepository. Trước create/update, lấy category managed từ DB bằng categoryId rồi gán vào Orchid. Dùng @Transactional cho write operations.
Kết quả mong đợi	POST/PUT không tạo category “rác”; Orchid luôn trỏ tới category tồn tại.
Cách kiểm tra	Test với categoryId không tồn tại và xác nhận request bị từ chối.
Lỗi thường gặp	Gọi save trực tiếp với object category chưa được resolve; nhét logic lookup vào controller.

Trạng thái project: V5 – Business/persistence layer hoàn chỉnh.

OrchidService.java – implementation tham chiếu

```
package com.example.orchid.services;

import com.example.orchid.pojos.Orchid;
import com.example.orchid.pojos.OrchidCategory;
import com.example.orchid.repositories.IOrchidCategoryRepository;
import com.example.orchid.repositories.IOrchidRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Optional;

@Service
@Transactional(readOnly = true)
public class OrchidService implements IOrchidService {
    private final IOrchidRepository orchidRepository;
    private final IOrchidCategoryRepository categoryRepository;

    public OrchidService(IOrchidRepository orchidRepository,
                        IOrchidCategoryRepository categoryRepository) {
        this.orchidRepository = orchidRepository;
        this.categoryRepository = categoryRepository;
    }

    @Override
    public List<Orchid> getAll() { return orchidRepository.findAll(); }

    @Override
    public List<Orchid> searchByName(String name) {
        return orchidRepository.findByOrchidNameContainingIgnoreCase(name);
    }

    @Override
```

```

public Optional<Orchid> getById(Long id) { return orchidRepository.findById(id); }

private OrchidCategory resolveCategory(Orchid orchid) {
    if (orchid.getOrchidCategory() == null ||
        orchid.getOrchidCategory().getCategoryId() == null) {
        throw new IllegalArgumentException("categoryId is required");
    }
    Long categoryId = orchid.getOrchidCategory().getCategoryId();
    return categoryRepository.findById(categoryId)
        .orElseThrow(() -> new IllegalArgumentException("Category not found: " + categoryId));
}

@Override
@Transactional
public Orchid create(Orchid orchid) {
    orchid.setOrchidID(null);
    orchid.setOrchidCategory(resolveCategory(orchid));
    return orchidRepository.save(orchid);
}

@Override
@Transactional
public Optional<Orchid> update(Long id, Orchid input) {
    return orchidRepository.findById(id).map(existing -> {
        existing.setOrchidName(input.getOrchidName());
        existing.setIsNatural(input.getIsNatural());
        existing.setOrchidDescription(input.getOrchidDescription());
        existing.setIsAttractive(input.getIsAttractive());
        existing.setOrchidURL(input.getOrchidURL());
        existing.setOrchidCategory(resolveCategory(input));
        return orchidRepository.save(existing);
    });
}

@Override
@Transactional
public boolean delete(Long id) {
    if (!orchidRepository.existsById(id)) return false;
    orchidRepository.deleteById(id);
    return true;
}
}

```

Mẹo: Nếu dùng `IllegalArgumentException` như mẫu học tập, Controller cần chuyển lỗi category thành 400. Trong project production, nên có exception type + global exception handler riêng.

Bước 13. Implement OrchidController

Mục	Nội dung
Mục tiêu	Expose service qua REST endpoints có method/path/status rõ ràng.
Thao tác	Tạo <code>@RestController @RequestMapping("/api/orchids")</code> . Implement GET list/search, GET id, POST, PUT, DELETE.
Kết quả mong đợi	Controller không gọi repository trực tiếp; response dùng status 200/201/204/400/404 phù hợp.
Cách kiểm tra	Đọc method signatures và API contract; compile project.
Lỗi thường gặp	Đặt business rule trong controller; trả 200 cho mọi trường hợp; bỏ <code>@RequestBody/@PathVariable</code> .

Trạng thái project: V6 – REST layer hoàn chỉnh.

OrchidController.java – implementation tham chiếu

```

package com.example.orchid.controllers;

import com.example.orchid.pojos.Orchid;
import com.example.orchid.services.IOrchidService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

```

```

import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/orchids")
public class OrchidController {
    private final IOrchidService orchidService;

    public OrchidController(IOrchidService orchidService) {
        this.orchidService = orchidService;
    }

    @GetMapping
    public List<Orchid> getAll(@RequestParam(required = false) String name) {
        if (name != null && !name.isBlank()) return orchidService.searchByName(name);
        return orchidService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Orchid> getById(@PathVariable Long id) {
        return orchidService.getById(id)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<?> create(@RequestBody Orchid orchid) {
        try {
            return ResponseEntity.status(HttpStatus.CREATED).body(orchidService.create(orchid));
        } catch (IllegalArgumentException ex) {
            return ResponseEntity.badRequest().body(ex.getMessage());
        }
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id, @RequestBody Orchid orchid) {
        try {
            var updated = orchidService.update(id, orchid);
            if (updated.isEmpty()) return ResponseEntity.notFound().build();
            return ResponseEntity.ok(updated.get());
        } catch (IllegalArgumentException ex) {
            return ResponseEntity.badRequest().body(ex.getMessage());
        }
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return orchidService.delete(id)
            ? ResponseEntity.noContent().build()
            : ResponseEntity.notFound().build();
    }
}

```

Bước 14. Chạy app và verify DDL/SQL/schema

Mục	Nội dung
Mục tiêu	Chứng minh JPA mapping tạo đúng relational schema.
Thao tác	Run OrchidApplication. Đọc console. Mở SQL Server Object Explorer/SSMS để xem tables orchids, orchid_categories và foreign key.
Kết quả mong đợi	Không có startup error; schema có PK/FK đúng; SQL log xuất hiện.
Cách kiểm tra	Chụp evidence console + database schema vào evidence/console và evidence/sqlserver.
Lỗi thường gặp	Chỉ nhìn dòng Started Application mà không kiểm tra schema; connect nhầm database.

Trạng thái project: V7 – Persistence infrastructure verified.

Mẹo: Một ứng dụng “start thành công” chưa chứng minh mapping đúng. Phải kiểm tra table, column, PK/FK và SQL log.

SQL seed – tạo Category tối thiểu cho các test POST/PUT

```
INSERT INTO orchid_categories (category_name)
VALUES ('Cattleya'), ('Dendrobium');

SELECT category_id, category_name
FROM orchid_categories
ORDER BY category_id;
```

Precondition cho CRUD: Ghi lại ID thực tế của Cattleya và Dendrobium. Các body mẫu phía dưới giả định lần tạo mới trên database sạch cho ID 1 và 2; nếu ID thực tế khác, phải dùng ID thật.

Bước 15. Tạo Postman collection và API contract

Mục	Nội dung
Mục tiêu	Tổ chức API tests và làm rõ contract trước CRUD.
Thao tác	Tạo collection SBA301 Slot18 Orchid API, variable baseUrl=http://localhost:8080. Tạo docs/api-contract.md.
Kết quả mong đợi	Collection chứa các request theo test matrix; API contract ghi method/path/body/status.
Cách kiểm tra	URL resolve đúng {{baseUrl}}/api/orchids.
Lỗi thường gặp	BaseUrl thừa /; dùng path /orchids theo Lab cũ nhưng controller mới là /api/orchids.

Trạng thái project: V7 – Test workspace + contract sẵn sàng.

API name	HTTP	Path	Expected
GET Orchids	GET	{{baseUrl}}/api/orchids	200
Search Orchids	GET	{{baseUrl}}/api/orchids?name=cat	200
GET Orchid	GET	{{baseUrl}}/api/orchids/{id}	200 / 404
POST Orchid	POST	{{baseUrl}}/api/orchids	201 / 400
PUT Orchid	PUT	{{baseUrl}}/api/orchids/{id}	200 / 400 / 404
DELETE Orchid	DELETE	{{baseUrl}}/api/orchids/{id}	204 / 404

Bước 16. Kiểm tra GET list và search

Mục	Nội dung
Mục tiêu	Kiểm chứng GET collection và @RequestParam derived query.
Thao tác	Sau khi có dữ liệu seed (hoặc POST ở bước 18 rồi quay lại), gửi GET /api/orchids và GET /api/orchids?name=cat.
Kết quả mong đợi	Status 200; list trả array JSON; search chỉ chứa tên phù hợp.
Cách kiểm tra	Quan sát SQL log: findAll và query theo orchidName.
Lỗi thường gặp	Nhầm query param thành path param; method repository sai property.

Trạng thái project: V8 – GET list/search verified.

Bước 17. Kiểm tra GET detail và missing resource

Mục	Nội dung
-----	----------

Mục	Nội dung
Mục tiêu	Phân biệt resource tồn tại và resource không tồn tại.
Thao tác	GET /api/orchids/{existingId}; sau đó GET /api/orchids/999999.
Kết quả mong đợi	Existing → 200 + JSON object; missing → 404.
Cách kiểm tra	Chụp method + URL + status + body. Ghi vào test-matrix.md.
Lỗi thường gặp	Trả null với 200; hiểu 404 là database down.

Trạng thái project: V8 – GET detail/negative verified.

Bước 18. POST – Tạo Orchid mới và verify relationship

Mục	Nội dung
Mục tiêu	Tạo resource có foreign key hợp lệ và kiểm tra state server/database thay đổi.
Thao tác	Đảm bảo categoryId=1 tồn tại. POST JSON mẫu. Sau đó GET resource và kiểm tra row orchids.category_id trong SQL Server.
Kết quả mong đợi	POST → 201; response có ID mới; category nested có categoryId; DB row có FK đúng.
Cách kiểm tra	Postman evidence + SELECT evidence + GET hậu điều kiện.
Lỗi thường gặp	Gửi categoryId không tồn tại; body dùng orchidCategory dưới dạng string; tự gán orchidID.

Trạng thái project: V8 – POST + relationship verified.

POST body – relationship qua categoryId

```
{
  "orchidName": "Cattleya Queen",
  "isNatural": true,
  "orchidDescription": "Demo orchid for Slot 18",
  "orchidCategory": { "categoryId": 1 },
  "isAttractive": true,
  "orchidURL": "https://example.com/orchid.jpg"
}
```

Mẹo: Sau POST, verification bắt buộc gồm cả API response và database state. Chỉ có 201 chưa đủ chứng minh persistence/relationship đúng.

Bước 19. PUT – Cập nhật Orchid và verify

Mục	Nội dung
Mục tiêu	Củng cố full update và relationship replacement.
Thao tác	PUT /api/orchids/{id} với body đầy đủ. Có thể đổi categoryId sang category khác tồn tại.
Kết quả mong đợi	Status 200; GET lại thấy tất cả field theo trạng thái mới; DB foreign key được cập nhật.
Cách kiểm tra	So sánh before/after ở Postman và SQL Server.
Lỗi thường gặp	Gửi body partial nhưng kỳ vọng PUT giữ các field cũ; đổi category sang ID không tồn tại.

Trạng thái project: V8 – PUT verified.

PUT body – ví dụ

```
{
```

```

"orchidName": "Cattleya Queen Updated",
"isNatural": false,
"orchidDescription": "Updated by PUT",
"orchidCategory": { "categoryId": 2 },
"isAttractive": true,
"orchidURL": "https://example.com/orchid-updated.jpg"
}

```

Bước 20. DELETE – Xóa Orchid và kiểm tra hậu điều kiện

Mục	Nội dung
Mục tiêu	Hiểu delete success phải được xác minh bằng state sau thao tác.
Thao tác	DELETE /api/orchids/{id}; sau đó GET cùng id và kiểm tra DB.
Kết quả mong đợi	DELETE → 204; GET sau DELETE → 404; row không còn trong SQL Server.
Cách kiểm tra	Lưu evidence của DELETE và GET hậu điều kiện.
Lỗi thường gặp	Chỉ nhìn status 204; xóa nhầm category; kỳ vọng response body với 204.

Trạng thái project: V8 – CRUD đầy đủ đã verify.

Mẹo: Pattern test bắt buộc: POST → GET lại; PUT → GET lại; DELETE → GET lại.

Bước 21. Break-It Lab – cố tình tạo lỗi JPA/REST

Mục	Nội dung
Mục tiêu	Học debug từ triệu chứng → layer → nguyên nhân → fix → regression test.
Thao tác	Chọn ít nhất 4 lỗi trong bảng Break-It bên dưới. Mỗi lỗi ghi Expected failure, Actual symptom, Root cause, Fix, Retest.
Kết quả mong đợi	Có evidence trước/sau; sinh viên xác định được layer lỗi.
Cách kiểm tra	Sau khi sửa, full CRUD regression vẫn pass.
Lỗi thường gặp	Sửa mò nhiều file cùng lúc; xóa log; không ghi lại root cause.

Trạng thái project: V9 – Debugging evidence hoàn chỉnh.

Break case	Thay đổi có chủ đích	Triệu chứng cần quan sát	Layer
B1	Sai databaseName/port	Datasource startup failure	Configuration
B2	Bỏ @Id khỏi Orchid	Entity metadata/startup failure	JPA mapping
B3	mappedBy="category_id"	Unknown mappedBy property	Relationship
B4	Đổi repository ID type thành String	Type mismatch/runtime behavior	Repository
B5	POST categoryId không tồn tại	400 theo service rule	Service/business
B6	GET id không tồn tại	404	Controller/API
B7	Sai Content-Type/body JSON	400 request parsing	HTTP/Jackson
B8	Đổi @RequestMapping path	404 route mismatch	Controller/routing

Bước 22. Lập Test Matrix và Evidence Package

Mục	Nội dung
-----	----------

Mục	Nội dung
Mục tiêu	Biến thao tác thủ công thành bộ kiểm chứng có thể audit.
Thao tác	Tạo docs/test-matrix.md. Ghi test ID, precondition, request, expected, actual, pass/fail, evidence file.
Kết quả mong đợi	Có ít nhất 10 test: list, search, detail, missing, POST valid/invalid category, PUT valid/missing, DELETE valid/missing.
Cách kiểm tra	Đối chiếu từng evidence filename với test ID.
Lỗi thường gặp	Ảnh không thấy status/URL; test matrix chỉ ghi PASS nhưng không có actual.

Trạng thái project: V9 – Verification package hoàn chỉnh.

Bước 23. Viết README và concept trace

Mục	Nội dung
Mục tiêu	Chứng minh khả năng tái lập và hiểu kiến trúc.
Thao tác	README: prerequisites, DB setup, config, run, endpoints, test, known issues. concept-trace.md: mô tả một POST end-to-end và một GET end-to-end.
Kết quả mong đợi	Người khác có thể clone/config/run/test; trace nêu đủ Controller→Service→Repository→Hibernate→SQL Server→Response.
Cách kiểm tra	Đọc README từ đầu trên clean terminal; kiểm tra không lộ credential.
Lỗi thường gặp	README chỉ mô tả code; thiếu lệnh/run config; concept trace copy định nghĩa chung.

Trạng thái project: V10 – Documentation hoàn chỉnh.

Bước 24. Final verification và Git checkpoint

Mục	Nội dung
Mục tiêu	Đóng bài ở trạng thái reproducible, có bằng chứng và có thể phản biện.
Thao tác	Chạy clean test/build, run app, execute mandatory test set, kiểm tra evidence/docs, git diff, commit/tag.
Kết quả mong đợi	mvn clean test PASS; mandatory tests PASS; Git clean; package không chứa credential thật.
Cách kiểm tra	Dùng checklist Phần 9 + Human Verification Gate.
Lỗi thường gặp	Commit node_modules/target/log dump; quên xóa password; evidence không map test case.

Trạng thái project: V10 – Lab 04 ready for submission.

Final commands

```
mvn clean test
git status
git add .
git commit -m "Complete SBA301 Slot18 Lab04 JPA REST"
git tag s18-lab04-complete
```

8. ĐÁNH GIÁ KẾT QUẢ HOÀN THÀNH TỪNG BƯỚC

Milestone	Điều kiện PASS	Evidence tối thiểu
V0	Baseline build/test ổn định	Console + Git status

Milestone	Điều kiện PASS	Evidence tối thiểu
V1	Spring Boot skeleton + dependencies đúng	pom.xml + build PASS
V2	SQL Server kết nối + structure đúng	Startup log + config redacted
V3	Entity + 1–N/N–1 mapping đúng	Code + ER/FK sketch
V4	JpaRepository + derived query	Repository code
V5	Service + transaction + category validation	Service code + negative test
V6	REST Controller contract hoàn chỉnh	Controller + endpoint table
V7	App run + schema/SQL verified	Console + SQL Server evidence
V8	CRUD/search/negative tests pass	Postman + DB postconditions
V9	Break-It + test matrix hoàn chỉnh	Before/after + root cause
V10	Docs + clean verification + Git checkpoint	README + concept trace + clean build

Quy tắc PASS: Không chấp nhận “code nhìn đúng” thay cho evidence chạy thật. Một milestone chỉ PASS khi có kết quả thực thi hoặc bằng chứng cấu trúc tương ứng.

9. CHẠY VÀ KIỂM TRA BÀI THỰC HÀNH HOÀN CHỈNH

9.1 Quy trình chạy từ clean state

- 1) SQL Server chạy và OrchidDB tồn tại.
- 2) Cập nhật credential cục bộ; không commit secret.
- 3) mvn clean test.
- 4) Run OrchidApplication.
- 5) Kiểm tra schema PK/FK và console SQL.
- 6) Chạy Postman collection theo thứ tự test matrix.
- 7) Kiểm tra hậu điều kiện trong database.
- 8) Chạy lại negative tests.
- 9) Chụp/đặt tên evidence theo Test ID.
- 10) git status phải sạch sau commit cuối.

9.2 Bộ test case bắt buộc

ID	Test	Expected
T01	GET /api/orchids	200 + JSON array
T02	GET /api/orchids?name=<keyword>	200 + filtered array
T03	GET existing id	200 + JSON object
T04	GET missing id	404
T05	POST valid category	201 + generated orchidID
T06	POST missing/nonexistent category	400
T07	PUT existing id + valid category	200 + updated state
T08	PUT missing id	404
T09	DELETE existing id	204 + GET after = 404

ID	Test	Expected
T10	DELETE missing id	404
T11	DB verify category_id FK	Row points to expected category
T12	Restart application then GET created data	Data remains in SQL Server

9.3 Câu hỏi kiểm tra miệng sau test

- Khi POST thành công, component nào thực sự gửi SQL INSERT xuống database?
- Vì sao Controller không nên gọi JpaRepository trực tiếp trong bài này?
- mappedBy="orchidCategory" trỏ đến cái gì? Tại sao không phải "category_id"?
- Nếu categoryId không tồn tại, nên phát hiện ở layer nào? Vì sao?
- Status 204 khác 200 như thế nào với DELETE?
- Tại sao phải GET lại hoặc SELECT DB sau POST/PUT/DELETE?
- Spring Data tạo implementation của IOrchidRepository khi nào?
- Nếu app restart mà dữ liệu còn, điều đó chứng minh điểm gì so với in-memory repository?

10. LỖI THƯỜNG GẶP VÀ CÁCH XỬ LÝ

Triệu chứng	Nguyên nhân khả dĩ	Cách kiểm tra / xử lý
Failed to configure DataSource	URL/user/password/driver sai	Đọc root cause; test SQL Server connection; kiểm tra pom.xml
Login failed for user	Credential/quyền DB	Đăng nhập bằng cùng credential trong SSMS
Connection refused / timeout	SQL Server/port/TCP-IP	Kiểm tra service, port 1433/instance config
Not a managed type	Entity ngoài scan hoặc thiếu @Entity	Kiểm tra package root và annotation
No property ... found	Derived query method sai property	Đối chiếu tên field chính xác trong Entity
mappedBy reference unknown property	mappedBy dùng column thay vì Java field	Dùng mappedBy="orchidCategory"
Infinite JSON recursion	Serialize bidirectional graph	@JsonIgnore inverse collection hoặc DTO strategy
POST category invalid	Category chưa tồn tại	Seed category / dùng ID đúng / xử lý 400
404 mọi endpoint	RequestMapping/path/port sai	Đọc startup log + controller path + Postman URL
500 khi POST/PUT	Business/mapping/constraint exception	Đọc first meaningful cause; không chỉ nhìn stack trace cuối
DELETE 204 nhưng vẫn thấy data	Check nhầm DB/id hoặc transaction chưa đúng	GET lại + SELECT đúng database
Password xuất hiện trong Git	Config secret bị commit	Rotate secret nếu cần; remove khỏi commit; dùng env/local config

11. PHẦN MỞ RỘNG VỚI AI ASSISTANT

Nguyên tắc: AI được dùng để giải thích, tạo giả thuyết debug, review mapping và đề xuất test. AI không thay thế việc chạy code, đọc SQL log, kiểm tra database và giải thích bằng lời của sinh viên.

11.1 Prompt giải thích – 10 mẫu

1. Giải thích request POST /api/orchids đi qua Controller, Service, Repository, Hibernate và SQL Server như thế nào.
2. Phân biệt JPA, Hibernate và Spring Data JPA bằng chính code của Lab 04 này.
3. Tại sao Orchid là owning side trong quan hệ Many-to-One với OrchidCategory?
4. Giải thích mappedBy bằng field orchidCategory và foreign key category_id.

5. Tại sao JpaRepository không cần OrchidRepositoryImpl cho CRUD cơ bản?
6. Giải thích @Transactional(readOnly=true) ở class và @Transactional ở create/update/delete.
7. Phân biệt HTTP 400, 404 và 500 trong các tình huống của Lab 04.
8. Vì sao trả entity bidirectional trực tiếp có thể gây recursion khi serialize JSON?
9. Giải thích PUT trong lab này và điều gì xảy ra nếu gửi body thiếu field.
10. Giải thích vì sao verification phải kiểm tra cả Postman response và SQL Server state.

11.2 Prompt debug / code review – 10 mẫu

1. Tôi nhận lỗi Not a managed type. Hãy cho danh sách nguyên nhân theo thứ tự kiểm tra, không viết lại toàn project.
2. Tôi nhận lỗi mappedBy references an unknown target entity property. Hãy giải thích cách kiểm tra field và column name.
3. Review hai entity Orchid/OrchidCategory và chỉ ra owning side, inverse side, FK và rủi ro cascade.
4. Review IOrchidRepository.findByOrchidNameContainingIgnoreCase và giải thích cách Spring Data parse method name.
5. Controller đang trả 200 khi id không tồn tại. Hãy chỉ ra chỗ cần thay đổi và status phù hợp.
6. POST bị 500 khi categoryId không tồn tại. Hãy đề xuất cách biến lỗi business thành 400 mà không gọi repository từ controller.
7. API bị infinite recursion khi trả JSON. Hãy nêu 3 chiến lược xử lý và trade-off.
8. Tôi có 204 khi DELETE. Hãy đề xuất hậu điều kiện cần test để chắc dữ liệu thực sự bị xóa.
9. Hãy tạo 5 negative tests cho REST + JPA lab này nhưng không thêm security.
10. Hãy review test matrix của tôi và chỉ ra test nào chưa chứng minh persistence/relationship.

11.3 AI Verification Log

AI suggestion	Tôi đã kiểm tra bằng gì?	Kết quả	Accept/Reject + lý do
Ví dụ: mappedBy nên là category_id	Entity field + startup test	Sai	Reject: mappedBy dùng Java field orchidCategory

12. CHECKLIST Củng cố kiến thức SLOT 17–18

Nhóm	Tôi làm được nếu...	Self-check
Entity	Tôi tự tạo entity có PK/generation/column constraints và giải thích annotation.	☐
Relationship	Tôi xác định owning side/inverse side, mappedBy và FK.	☐
Repository	Tôi dùng JpaRepository và viết derived query đúng property.	☐
Service	Tôi đặt business rule/transaction ở Service thay vì Controller.	☐
Controller	Tôi mapping method/path/body/path variable/query param đúng.	☐
Status code	Tôi phân biệt 200/201/204/400/404.	☐
Persistence	Tôi chứng minh data còn sau restart và kiểm tra SQL Server state.	☐
Debug	Tôi khoanh vùng lỗi theo Configuration/JPA/Repository/Service/Controller/HTTP.	☐
Verification	Tôi có positive + negative + postcondition evidence.	☐
Explain	Tôi trace được một request end-to-end không cần đọc tài liệu.	☐

12.1 Human Verification Gate – 20 câu

1. JPA là specification hay implementation?
2. Hibernate đóng vai trò gì trong project này?
3. Spring Data JPA bổ sung abstraction gì?
4. OrchidCategory.orchids có mappedBy="orchidCategory" nghĩa là gì?
5. Foreign key category_id nằm ở table nào và vì sao?
6. Nếu đổi mappedBy thành category_id, lỗi conceptual là gì?
7. Tại sao repository generic ID phải là Long?
8. Tại sao create() phải resolve category từ repository?
9. Tại sao service không nên trả ResponseEntity?
10. Tại sao controller không nên chứa SQL/data access?
11. @RequestBody dùng cho dữ liệu nào?
12. @PathVariable khác @RequestParam thế nào trong chính API này?
13. POST thành công nên trả status nào? Vì sao?
14. DELETE thành công không có body nên dùng status nào?
15. GET missing id khác database connection failure thế nào?
16. Khi nào transaction của create() bắt đầu/kết thúc về mặt khái niệm?
17. Tại sao test POST phải GET lại?
18. Tại sao còn cần SELECT DB nếu Postman đã trả JSON?
19. Nêu một rủi ro của bidirectional relationship khi serialize JSON.
20. Nếu AI đề xuất annotation khác, bạn sẽ kiểm chứng bằng evidence nào?

13. RUBRIC ĐÁNH GIÁ BÀI TẬP TỔNG HỢP

Hạng mục	Điểm	Tiêu chí chính
A. Environment & SQL Server	10	Build/run đúng; datasource đúng; không lộ secret.
B. Entity & Relationship Mapping	20	Entity hợp lệ; PK/FK; ManyToOne/OneToMany; mappedBy/JoinColumn đúng.
C. Repository & Service	20	JpaRepository; derived query; service boundary; category validation; transaction.
D. REST Controller & API Contract	20	Endpoint/method/body/path/query/status đúng; controller không gọi repository trực tiếp.
E. CRUD + Negative + DB Verification	15	Bộ test bắt buộc; postcondition; DB/SQL evidence.
F. Break-It & Debugging	5	Ít nhất 4 lỗi có symptom/root cause/fix/retest.
G. Documentation & Concept Trace	5	README/test-matrix/concept-trace tái lập được.
H. Code quality & Git checkpoint	5	Naming/package rõ; clean build; commit/tag; không secret.
Tổng	100	

13.1 Mức đánh giá

Mức	Điểm	Mô tả
Excellent	90–100	Chạy đúng + mapping/relationship đúng + evidence mạnh + giải thích end-to-end rõ.
Good	75–89	Core CRUD/JPA đúng; còn thiếu một số negative/debug/evidence chi tiết.
Pass	60–74	API chạy và persistence cơ bản đúng nhưng relationship/verification/trace còn yếu.

Mức	Điểm	Mô tả
Needs revision	<60	Không tái lập được, sai layer/mapping nghiêm trọng, thiếu evidence hoặc không giải thích được code.

PHỤ LỤC A. QUICK REFERENCE

Annotation / API	Vai trò ngắn	Vị trí thường dùng
@Entity	Persistent entity	Class
@Id	Primary key	Entity field
@GeneratedValue	Sinh khóa	ID field
@Column	Column constraints/mapping	Entity field
@ManyToOne	Nhiều Orchid thuộc một Category	Orchid.orchidCategory
@OneToMany	Một Category có nhiều Orchid	Category.orchids
mappedBy	Chỉ inverse side; trỏ tới Java field owning side	@OneToMany
@JoinColumn	Định nghĩa FK column ở owning side	@ManyToOne
JpaRepository<T,ID>	CRUD + paging/sorting + JPA operations	Repository interface
@Service	Business/service bean	Service class
@Transactional	Transaction boundary	Service/write methods
@RestController	REST web controller	Controller class
@RequestMapping	Base/request mapping	Controller class/method
@GetMapping/@PostMapping/...	HTTP verb shortcut	Controller methods
@PathVariable	Value từ URI path	Method parameter
@RequestParam	Value từ query string	Method parameter
@RequestBody	JSON body → Java object	Method parameter
ResponseEntity	Status/headers/body control	Controller return type

PHỤ LỤC B. WORKSHEET – REQUEST → DATABASE TRACE

Checkpoint	POST /api/orchids – sinh viên điền
1. HTTP request	Method, URL, Content-Type, JSON body
2. Controller	Method nào nhận request? Annotation nào bind body?
3. Service	Business rule nào được kiểm tra? Transaction boundary ở đâu?
4. Repository	Repository method nào được gọi? Category lookup có xảy ra không?
5. Hibernate/JPA	Entity state/association nào được persist?
6. SQL	INSERT/SELECT/UPDATE nào quan sát được?
7. Database state	Row/PK/FK nào thay đổi?
8. HTTP response	Status, headers, JSON body

Checkpoint	POST /api/orchids – sinh viên điền
9. Postcondition	GET/SELECT nào chứng minh kết quả?

Bài kiểm tra trọng tâm: Nếu sinh viên chỉ nói “Controller gọi Service rồi lưu database” nhưng không giải thích được category lookup, managed relationship, JpaRepository/Hibernate và FK category_id thì chưa đạt mức hiểu end-to-end.

PHỤ LỤC C. MỞ RỘNG TÙY CHỌN

- C1 – PATCH partial update: bổ sung @PatchMapping và giải thích khác PUT.
- C2 – Pagination/Sorting: GET /api/orchids?page=0&size=5&sort=orchidName,asc bằng Pageable.
- C3 – DTO strategy: tách request/response DTO khỏi entity và giải quyết JSON relationship sạch hơn.
- C4 – @DataJpaTest: kiểm thử repository derived query và relationship bằng test slice.
- C5 – Group Project Bridge: chọn một cặp entity 1–N trong ERD nhóm và lặp lại quy trình Map → Persist → Query → Verify → Explain.

Mở rộng không thay thế core: Chỉ làm Appendix C sau khi T01–T12 và Human Verification Gate đã hoàn thành.